Avaloka AI Intern Assignment

Name: Gowthami Kanchi

Assignment: Data Structures & Systems Design

Problem 1: LRU Cache Implementation

Approach and Logic :

An LRU (Least Recently Used) Cache stores key-value pairs with a fixed capacity. When the cache is full, it removes the least recently used item to make space for a new item.

I use a hash map (Python dictionary) and a doubly linked list. The hash map provides average O(1) access to a node using its key. The doubly linked list maintains the usage order, with the most recently used item at the front and the least recently used item at the end.

Whenever `get()` accesses an existing key or `put()` updates a key, the corresponding node moves to the front. When the cache reaches capacity, the node at the end is removed.

This combination ensures that both `get()` and `put()` run in average O(1) time.

Python Implementation :

```
class Node:
    def __init__(self, key=None, value=None):
        self.key = key
        self.value = value
        self.prev = None
        self.next = None
class LRUCache:
    def __init__(self, capacity):
        if capacity < 0:
            raise ValueError("Capacity cannot be negative")
        self.capacity = capacity
        self.cache = {}
        self.head = Node()
        self.tail = Node()
        self.head.next = self.tail
        self.tail.prev = self.head
    def _remove(self, node):
        node.prev.next = node.next
        node.next.prev = node.prev
    def _add_to_front(self, node):
        node.next = self.head.next
        node.prev = self.head
```

```
        self.head.next.prev = node
        self.head.next = node
    def get(self, key):
        if key not in self.cache:
            return -1
        node = self.cache[key]
        self._remove(node)
        self._add_to_front(node)
        return node.value
    def put(self, key, value):
        if self.capacity == 0:
            return
        if key in self.cache:
            node = self.cache[key]
            node.value = value
            self._remove(node)
            self._add_to_front(node)
            return
        if len(self.cache) >= self.capacity:
            lru_node = self.tail.prev
            self._remove(lru_node)
            del self.cache[lru_node.key]
        node = Node(key, value)
        self.cache[key] = node
        self._add_to_front(node)
# Example
lru = LRUCache(2)
lru.put(1, 10)
lru.put(2, 20)
print(lru.get(1))
lru.put(3, 30)
print(lru.get(2))
print(lru.get(3))
```

Why did I choose these data structures?

A hash map allows quick access to cache entries by key, but it does not maintain the order of usage. A doubly linked list maintains the usage order and allows nodes to be removed or moved in constant time.

Using both structures avoids searching through the entire cache to find an entry or identify the least recently used item. The main trade-off is additional memory required for the linked-list pointers.

Problem 2: Event Scheduler

Approach and Logic

Each event is represented by a tuple containing its start and end times.

For `can_attend_all(events)`, I sort the events by start time and compare consecutive events. If an event ends after the next event starts, the events overlap, so attending all events is impossible. Otherwise, the person can attend all events.

For `min_rooms_required(events)`, I sort the events by start time and use a min-heap to store the end times of meetings currently occupying rooms. Before scheduling a meeting, I remove all end times that are less than or equal to its start time, because those rooms are available again. I then add the current meeting's end time to the heap.

The maximum number of simultaneously active meetings determines the minimum number of rooms required. The heap allows room availability to be checked efficiently.

Adjacent meetings are not considered overlapping, so a room can be reused when one meeting ends exactly when another begins.

Python Implementation :

```python
import heapq
def can_attend_all(events):
    if not events:
        return True
    sorted_events = sorted(events, key=lambda event: event[0])
    for i in range(1, len(sorted_events)):
        if sorted_events[i - 1][1] > sorted_events[i][0]:
            return False
    return True
def min_rooms_required(events):
    if not events:
        return 0
    sorted_events = sorted(events, key=lambda event: event[0])
    room_end_times = []
    for start, end in sorted_events:
        while room_end_times and room_end_times[0] <= start:
            heapq.heappop(room_end_times)
        heapq.heappush(room_end_times, end)
    return len(room_end_times)
# Example 1
events1 = [(9, 10), (10, 11), (11, 12)]
print(can_attend_all(events1))      # True
print(min_rooms_required(events1))  # 1
```

```
# Example 2
events2 = [(9, 12), (10, 11), (11, 13)]
print(can_attend_all(events2))      # False
print(min_rooms_required(events2))  # 2
```

Explain the room requirement calculation

Consider the events `[(9, 12), (10, 11), (11, 13)]`.

- The meeting from 9 to 12 requires the first room.
- The meeting from 10 to 11 overlaps with the first meeting, so a second room is needed.
- At 11, the second meeting ends. Its room can be reused for the meeting starting at 11.
- The first meeting continues until 12, so two rooms are still required overall.

Therefore, the minimum number of rooms required is 2.

Final Discussion & Analysis

1. Time and Space Complexity

Let n represent the number of events and c represent the LRU cache capacity.

Function	Time Complexity	Space Complexity
LRUCache.__init__	O(1)	O(1)
LRUCache.get()	Average O(1)	O(1) Auxiliary
LRUCache.put()	Average O(1)	O(1) auxiliary
Can_attend_all()	O(n log n)	O(n)
Min_rooms_required()	O(n log n)	O(n)

Explanation:

- LRU cache operations take average O(1) time because dictionary lookups and linked-list operations are constant time.
- `can_attend_all()` sorts the events in O(n log n) time and checks them in O(n) time.
- `min_rooms_required()` sorts the events and performs heap operations, resulting in O(n log n) time.

- The scheduler requires O(n) additional space for the sorted events and, for room calculation, the heap.

2. Trade-offs: Why combine a hash map and a doubly linked list?

A hash map provides fast access to a node using its key, but it does not track which entry was used most recently.

A doubly linked list maintains the order of usage and supports constant-time node removal and insertion, but searching for a particular key in the list would take \(O(n)\) time.

Combining them provides both fast lookup and efficient usage-order management. The trade-off is additional memory for the linked-list nodes and pointers.

3. Future Proofing: How would you assign specific room numbers to events?

I would use two min-heaps: one to store available room IDs and another to track occupied rooms by their meeting end times.

First, I would sort all events by start time. For each event, I would release rooms whose meetings have ended by the event's start time. If a room is available, I would assign it to the event; otherwise, I would create or allocate another room.

The occupied-room heap would store pairs such as `(end_time, room_id)`. After assigning a room, I would record the meeting's end time and room ID.

This approach would allow the scheduler to assign specific room numbers, reuse rooms efficiently, and prevent overlapping meetings from being assigned to the same room.

4. Concurrency: What changes would be required to make the LRU Cache thread-safe?

If multiple threads access the cache simultaneously, one thread might modify the linked list while another thread is reading or updating it. This could cause race conditions or inconsistent cache state.

To prevent this, I would introduce a mutex or reentrant lock around the cache operations. The lock would protect both the hash map and the linked list, ensuring that lookup, updates, node movement, and eviction happen atomically.

In the actual implementation, the lock would be part of the `LRUCache` class and used inside both `get()` and `put()`.

A single lock is simple and reliable, although it may reduce performance when many threads access the cache concurrently. More advanced implementations could use finer-grained locks, but they would be more complex to maintain correctly.